



华中科技大学

操作系统原理课程实验报告

姓 名：王家乐
学 院：计算机科学与技术学院
专 业：计算机科学与技术
班 级：计算机本硕博 2301 班
学 号：U202315763
指导教师：廖小飞

分数	
教师签名	

2025 年 12 月 16 日

目 录

实验一 打印用户程序调用栈(挑战实验 1)	- 1 -
1.1 实验目的	- 1 -
1.2 实验内容	- 1 -
1.2.1 函数调用链路	- 1 -
1.2.2 定位节的位置	- 2 -
1.2.3 寻找函数符号	- 4 -
1.2.4 打印用户程序调用栈	- 4 -
1.2.5 实验结果	- 6 -
1.2.6 日志 (git log)	- 6 -
1.3 实验调试及心得	- 6 -
实验二 复杂缺页异常(挑战实验 2)	- 7 -
2.1 实验目的	- 7 -
2.2 实验内容	- 7 -
2.2.1 相关实现	- 7 -
2.2.2 实验结果	- 8 -
2.2.3 日志 (git log)	- 8 -
2.3 实验调试及心得	- 9 -
实验三 进程等待和数据段复制(挑战实验 3)	- 10 -
3.1 实验目的	- 10 -
3.2 实验内容	- 10 -
3.2.1 进程等待	- 10 -

3.2.2 数据段复制	- 12 -
3.2.3 实验结果	- 13 -
3.2.4 日志 (git log)	- 13 -
3.3 实验调试及心得	- 13 -
实验四 相对路径(挑战实验 4)	- 15 -
4.1 实验目的	- 15 -
4.2 实验内容	- 15 -
4.2.1 显示当前目录 pwd	- 15 -
4.2.2 切换目录 cd	- 16 -
4.2.3 修改 sys_user_open	- 16 -
4.2.4 实验结果	- 17 -
4.2.5 日志 (git log)	- 18 -
4.3 实验调试及心得	- 18 -

实验一 打印用户程序调用栈(挑战实验 1)

1.1 实验目的

lab1_challenge1 的实验目的，是通过补全 PKE 内核的关键机制，把给定用户程序所形成的调用链(main -> f1 -> ... -> f8)在触发 print_backtrace(7) 时“回溯出来”，即从当前最深处开始向上按层打印用户态函数调用栈并得到预期输出（依次打印 f8 到 f2），同时支持用参数控制回溯深度并在超过实际深度时于 main 处停止，从而在实践中贯通系统调用进入内核后的处理路径、用户态/内核态栈切换带来的相关问题、以及如何根据用户栈中找到的返回地址将虚拟地址解析为源程序符号（通常需要利用 ELF 的符号信息如 .symtab/.strtab 等）并输出可读的函数名。

1.2 实验内容

1.2.1 函数调用链路

用户调用的函数有一个完整的调用链路，会转换为 S 态或者 M 态的系统调用。观察到 app_print_backtrace 调用了函数 print_backtrace，应该在文件 user_lib.h 中得到声明，所以首先声明该函数：

```
/* header file to be used by applications. */
int print_backtrace(int depth);
```

该函数应在 user_lib.c 中实现，而 user_lib.c 中所有函数最终都转化为了系统调用，添加一个新的系统调用号 SYS_user_print_backtrace，并将函数 print_backtrace 转换为系统调用：

```
int print_backtrace(int depth) {
    return do_user_call(SYS_user_print_backtrace, depth, 0, 0, 0, 0, 0, 0);
}
```

和其他系统调用一样，do_user_call 将受理这个调用，设置好当前进程的上下文后，由用户态（U）陷入监管态（S），并将控制权交给 handle_syscall。handle_syscall 将 a0~a7 寄存器的值交给函数 do_syscall，因此我们需要在函数 do_syscall 中再添加一个分支：

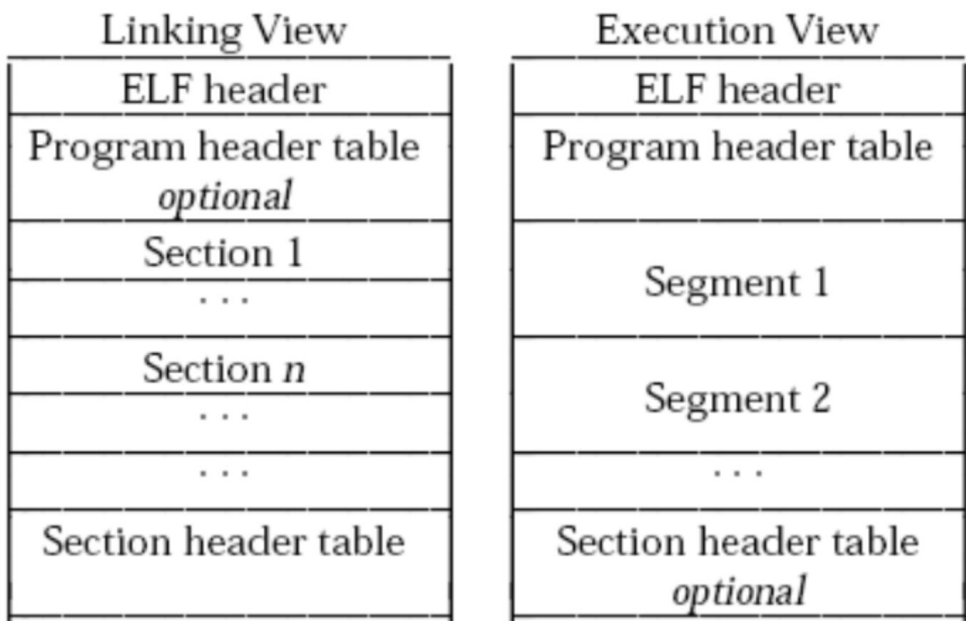
```
...
case SYS_user_print_backtrace:
    return sys_user_print_backtrace(a1);
```

当 `a0` (用户调用号) 为 `SYS_user_print_backtrace` 时, `sys_user_print_backtrace` 函数被调用, 将这一信号从 U 态交给 S 态相关函数执行。

1.2.2 定位节的位置

Challenge1 与基础实验不一样: 需要打印的调用栈函数名并不存放在当前函数的栈帧里, 而是记录在用户程序对应的 ELF 文件中。内核启动时只会把 ELF 的可装入段映射到内核虚拟内存, 其它段会被丢弃, 所以 `sys_user_print_backtrace` 函数无法直接拿到包含符号信息的 ELF 内容, 必须进一步深入处理。

要想把返回地址解析成可读的函数名, 必须把“符号信息的获取”从运行时回溯逻辑里剥离出来, 提前在装载阶段完成。在把程序段拷贝到内存并建立用户态地址空间的同时, 从 ELF 的 `section header table` 出发定位符号表与字符串表, 提取出函数符号的起始地址与大小, 并将其与函数名一并缓存到内核侧的固定结构中。这样一来, 符号信息不需要在运行时依赖原始 ELF 内容, `sys_user_print_backtrace` 只需要拿到当前进程 `trapframe` 中保存的用户态帧指针 (`s0`), 按栈帧链依次取出每一层的返回地址, 再在内核已缓存的符号数组中做区间匹配 (判断返回地址是否落在某个函数 `[st_value, st_value + st_size)` 范围内), 即可将调用栈从“地址序列”转换为“函数名序列”并逐层打印。



ELF 文件主要是由两个视图组成, 一个是链接时的视角, 还有一个执行时的视角。我们关注 ELF 的链接视图 (即外存上的可执行文件), 其中节头表 ELF header 是一个结构体 `elf_ssect_header`:

```
typedef struct elf_header_t {  
    uint32 magic;  
    uint8 elf[12];  
};
```

```

uint16 type;      /* Object file type */
uint16 machine;   /* Architecture */
uint32 version;   /* Object file version */
uint64 entry;     /* Entry point virtual address */
uint64 phoff;     /* Program header table file offset */
uint64 shoff;     /* Section header table file offset */
uint32 flags;     /* Processor-specific flags */
uint16 ehsize;    /* ELF header size in bytes */
uint16 phentsize; /* Program header table entry size */
uint16 phnum;     /* Program header table entry count */
uint16 shentsize; /* Section header table entry size */
uint16 shnum;     /* Section header table entry count */
uint16 shstrndx;  /* Section header string table index */
} elf_header;

```

在本实验中，主要有以下几个字段值比较重要。1. shoff、shentsize、shnum：描述节区头表的偏移、大小、数量(有多少个节首部)。2. shstrndx：这一项描述的是字符串表在节区头表中的索引，本实验中 shstrndx 值为 17。

elf_section_header 类型表示了一个节头表项中的具体内容，在实验代码中并没有给出，需要我们自己在 elf.h 文件中定义：

```

typedef struct elf_section_header_t {
    uint32 sh_name;      /* Section name, index in string tbl */
    uint32 sh_type;      /* Type of section */
    uint64 sh_flags;
    uint64 sh_addr;
    uint64 sh_offset;    /* Section file offset */
    uint64 sh_size;      /* Size of section in bytes */
    uint32 sh_link;
    uint32 sh_info;
    uint64 sh_addralign;
    uint64 sh_entsize;   /* Entry size if section holds table */
}elf_section_header;

```

每一个 elf_section_header 解释了该节的名称、类型、大小以及在整个 ELF 文件中的字节偏移位置等等。一个程序中有很多个节，在本实验中，我们需要重点关注.symtab 和.strtab 节以及.shstrtab 节。

在所有的节中，.shstrtab 节起了一个关键作用，这个节里面包含所有节的名称字符串（但函数名不在其中）。因此，我们首先需要获取这个节。获取的步骤是，先找到它的节头表的位置，再从 ELF 文件中载入这个节头。再根据字段 sh_offset 和 sh_size，可以读取.shstrtab 节的内容到 shstr_buf：

```

...
// 读取 section header 字符串表
elf_section_header shstr_header;

```

```
uint64 shstr_offset = context->ehdr.shoff +
                    context->ehdr.shstrndx * sizeof(elf_section_header);
elf_fpread(context, &shstr_header, sizeof(shstr_header), shstr_offset);
// 加载所有 section 名称字符串
char shstr_buf[shstr_header.sh_size];
elf_fpread(context, shstr_buf, shstr_header.sh_size, shstr_header.sh_offset);
```

1.2.3 寻找函数符号

现在我们已经可以通过节名在 ELF 文件中定位到任意一个节。具体做法是：先在 `shstr_buf` 里查找目标节名字符串（例如 `.text`、`.data` 等），从而得到该字符串在 `.shstrtab` 中的偏移；随后遍历所有节头表项，找到其 `name` 字段与该偏移量一致的那一项，该节头表项就对应目标节。确定了该节头后，就可以利用其 `offset` 字段定位该节在 ELF 文件中的起始位置，并用 `size` 字段得到该节的长度。基于此，我们就能把 `.symtab` 和 `.strtab` 两个节完整读入内存。其中 `.strtab` 与 `.shstrtab` 一样属于字符串表节，但用途不同：`.strtab` 保存的是各类符号（函数名、变量名等）的字符串名称；而 `.symtab` 则存放符号表本体，本质上是 `elf_symbol` 结构体构成的数组，每一项对应一个符号的元信息。

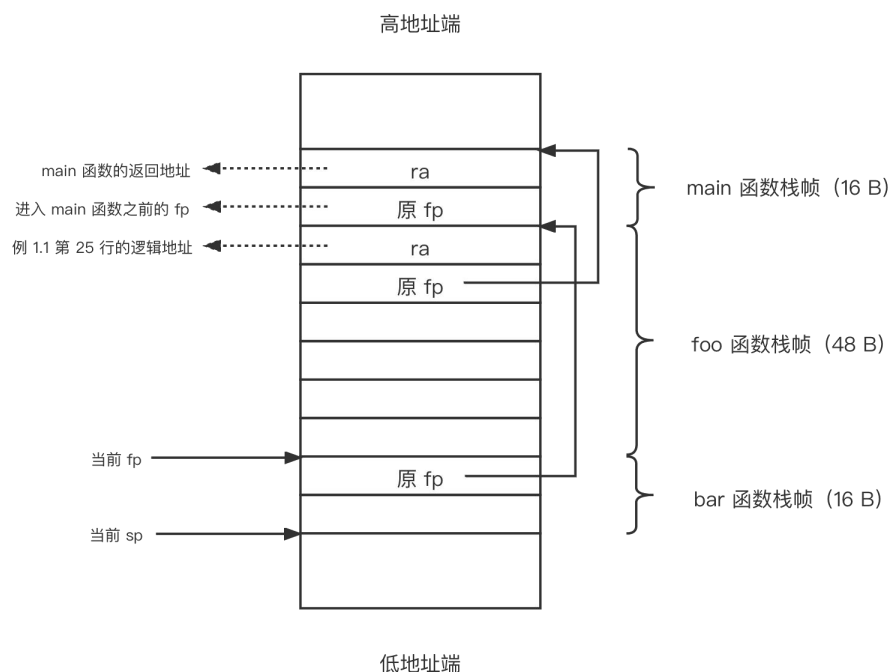
```
typedef struct elf_sym {
    uint32 st_name;      /* Symbol name, index in string tbl */
    unsigned char st_info; /* Type and binding attributes */
    unsigned char st_other; /* No defined meaning, 0 */
    uint16 st_shndx;     /* Associated section index */
    uint64 st_value;     /* Value of the symbol */
    uint64 st_size;     /* Associated symbol size */
} elf_symbol;
```

`st_name` 表示该符号在 `.strtab` 字符串表中的偏移位置，用于索引符号名称；`st_value` 记录符号对应的地址，对于函数符号而言即函数的入口地址；`st_size` 描述符号所占的大小，对函数来说等价于其生成的机器指令长度。因此，可以认为 `st_value + st_size - 1` 是函数机器指令 `ret` 的尾地址。基于这一信息，当已知某条指令的地址时，可以通过遍历 `.symtab` 中的 `elf_symbol` 项，判断该地址是否落在某个符号的地址区间内。一旦找到满足条件的符号条目，便可利用其 `st_name` 字段作为索引，从 `.strtab` 中读取对应的字符串，从而确定该指令所属函数的名称。

1.2.4 打印用户程序调用栈

我们已经可以根据实际进程中某个机器指令的地址，找到其所属的函数名称。接下来的问题是，如何找到这一系列的地址来打印调用栈呢？

下图是一个函数调用的栈帧实例，`main()` 函数调用函数 `foo()`，然后 `foo()` 又调用函数 `bar()`。发生函数调用时，调用者的 `epc + 4`（返回地址）会被压栈，同时基址寄存器 `fp` 也被压栈，然后进入被调用者的栈帧中。



可以把函数 `bar()` 当做 `print_traceback()`。那么 `print_traceback()` 在回溯调用栈时，首先应该打印函数 `foo()` 的名称，而这可以通过 `foo` 函数栈帧中的 `ra` 获取。随后应打印函数 `main()` 的名称，同样可以通过 `main()` 函数栈帧中的 `ra` 获取。需要注意的是，一开始上一个栈帧的 `fp` 位于 `fp - 8`，之后都是 `fp - 16`。最终在 `syscall.c` 中完成以下两个函数：

```
ssize_t locate_function_name(uint64 frame_pointer) {
    uint64 ret_addr = *((uint64 *) (frame_pointer - 8));
    for (int i = 0; i < sym_count; i++) {
        uint64 func_start = symbols[i].st_value;
        uint64 func_end = func_start + symbols[i].st_size;
        if (ret_addr < func_start || ret_addr >= func_end) continue;
        sprint("%s\n", sym_names[i]);
        return strcmp(sym_names[i], "main") == 0 ? 1 : 0;
    }
    return 0;
}

ssize_t sys_user_print_backtrace(uint64 num_frames) {
    uint64 fp = current->trapframe->regs.s0;
    uint64 cur_fp = *((uint64 *) (fp - 8));
    for (int cnt = 0; cnt < num_frames; cnt++) {
        if (locate_function_name(cur_fp)) return 0;
        cur_fp = *((uint64 *) (cur_fp - 16));
    }
    return 0;
}
```

1.2.5 实验结果

```
huster@huster-VMware-Virtual-Platform:~/桌面/OS/riscv-pke$ spike ./obj/riscv-pke ./obj/app_print_backtrace
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
Application: ./obj/app_print_backtrace
Application program entry point (virtual address): 0x00000000810000a2
Switch to user mode...
back trace the user app in the following:
f8
f7
f6
f5
f4
f3
f2
User exit with code:0.
System is shutting down with exit code 0.
```

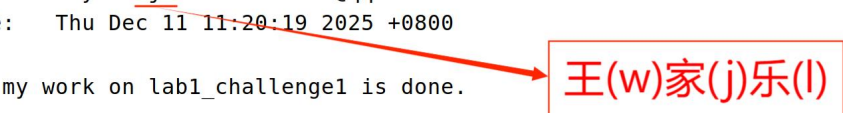
1.2.6 日志 (git log)

```
huster@huster-VMware-Virtual-Platform:~/桌面/OS/riscv-pke$ git log
commit 3fe70572b78509a765651f446b4c43fbd73a4c9 (HEAD -> lab1_challenge1_backtrace)
Author: Nuyoahwjl <3508529368@qq.com>
Date: Thu Dec 11 11:20:19 2025 +0800

    my work on lab1_challenge1 is done.

commit 60d48d57a4d118f91e4ac9e041f47dc48f2c7cf5
Merge: ff745a9 1dc7360
Author: Nuyoahwjl <3508529368@qq.com>
Date: Wed Dec 10 23:55:57 2025 +0800

    continue to work on lab1_challenge1
```



1.3 实验调试及心得

本实验最花时间的点主要在“地址—符号”的闭环验证：一方面要确保回溯链路上拿到的返回地址确实来自用户栈帧，另一方面要确保符号解析逻辑能把这些地址稳定地命中函数区间并输出预期的函数名序列。实际调试中我先只打印回溯出来的 `ret_addr` 十六进制，再用 `readelf/objdump` 核对 `ret_addr` 是否落在 `.text` 的合理范围，确认“地址正确”后再接入符号匹配和字符串打印。在实现上我采用了通过 `fp` 链逐层取 `ra` 并在到达 `main` 时终止的方式，回溯核心逻辑集中在 `locate_function_name` 与 `sys_user_print_backtrace` 两个函数里，并用参数控制回溯深度，避免越界回溯导致的错误访问。

实验二 复杂缺页异常(挑战实验 2)

2.1 实验目的

lab2_challenge1 的实验目的，是在继承 lab2_3 缺页处理能力的基础上，进一步修改 PKE 内核，让内核能够区分并分别处理不同成因的缺页异常。对给定 n 计算 0 到 n 的和，但要求将每一步递归的结果保存在数组 `ans` 中。创建数组时，使用 `malloc` 函数申请了一个页面（4KB）的大小，对应可以存储的个数上限为 1024。在函数调用时，试图计算 1025 求和，首先由于 n 足够大，所以在函数递归执行时会触发用户栈的缺页，需要对其进行正确处理，确保程序正确运行；其次，1025 在最后一次计算时会访问数组越界地址，由于该处虚拟地址尚未有对应的物理地址映射，因此属于非法地址的访问，这是不被允许的，对于这种缺页异常，应该提示用户并退出程序执行。

2.2 实验内容

2.2.1 相关实现

根据 lab2_3，我们知道，正常的缺页异常出现在以 `USER_STACK_TOP` 为起点向后增长的用户栈扩展区，这意味着程序需要新的栈页，因此内核应当为其补齐并映射相应的物理页供继续执行。而 `naive_malloc()` 的系统调用最终进入 `sys_user_allocate_page()`，其分配的用户内存是从 `g_ufree_page` 指向的空闲区域开始提供的，而非来自栈扩展区的按需增长。

```
// maybe, the simplest implementation of malloc in the world ... added @lab2_2
uint64 sys_user_allocate_page() {
    void* pa = alloc_page();
    uint64 va = g_ufree_page;
    g_ufree_page += PGSIZE;
    user_vm_map((pagetable_t)current->pagetable, va, PGSIZE, (uint64)pa,
                prot_to_type(PROT_WRITE | PROT_READ, 1));
    return va;
}
```

考虑到上述信息，我们可以在进行缺页异常处理的时候判断引起缺页异常的地址所在位置，如果该地址在 `naive_malloc()` 函数分配的空间部分，那么这部分地址空间理应是禁止访问的，所以程序输出错误信息然后退出即可。在 `stracp.c` 的 `handle_user_page_fault()` 增加：

```
// the page fault handler. added @lab2_3. parameters:
```

```
// stval: the virtual address that causes pagefault when being accessed.
void handle_user_page_fault(uint64 mcause, uint64 sepc, uint64 stval) {
    ...
    switch (mcause) {
        case CAUSE_STORE_PAGE_FAULT:
            if(stval >= g_ufree_page && stval < g_ufree_page+PGSIZE) {
                sprintf("this address is not available!\n");
                shutdown(-1);
                break;
            }
            ...
    }
}
```

2.2.2 实验结果

```
huster@huster-VMware-Virtual-Platform:~/桌面/OS/riscv-pke$ spike ./obj/riscv-pke ./obj/app_sum_sequence
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
PKE kernel start 0x0000000080000000, PKE kernel end: 0x0000000080007000, PKE kernel size: 0x0000000000007000 .
free physical memory address: [0x0000000080007000, 0x0000000087ffffff]
kernel memory manager is initializing ...
KERN_BASE 0x0000000080000000
physical address of _etext is: 0x0000000080004000
kernel page table is on
User application is loading.
user frame 0x0000000087fbc000, user stack 0x000000007ffff000, user kstack 0x0000000087fbb000
Application: ./obj/app_sum_sequence
Application program entry point (virtual address): 0x00000000000100a2
Switch to user mode...
handle_page_fault: 000000007ffdf8
handle_page_fault: 000000007ffcf8
handle_page_fault: 000000007ffbf8
handle_page_fault: 000000007ffa8
handle_page_fault: 000000007fff9f8
handle_page_fault: 000000007fff8f8
handle_page_fault: 000000007fff7f8
handle_page_fault: 000000007fff6f8
handle_page_fault: 000000000401000
this address is not available!
System is shutting down with exit code -1.
```

2.2.3 日志 (git log)

```
huster@huster-VMware-Virtual-Platform:~/桌面/OS/riscv-pke$ git log
commit 7fd31f2e99c3ec59254a28b7b232d7b57c4ab49f (HEAD -> lab2_challenge1_pagefaults)
Author: Nuyoahwjl <3508529368@qq.com>
Date: Thu Dec 11 21:23:17 2025 +0800

    my work on lab2_challenge1 is done.

commit 16b8632dcc64aeb09a65c95ef5969abef9d98931
Merge: be365de e32e344
Author: Nuyoahwjl <3508529368@qq.com>
Date: Thu Dec 11 19:39:47 2025 +0800

    continue to work on lab2_challenge1
```

2.3 实验调试及心得

本实验的关键是把“可恢复的缺页”（用户栈按需增长）与“不可恢复的缺页”（越界/非法地址访问）区分开，并在不可恢复场景下给出明确错误并退出。调试时我先把所有 `page fault` 的 `mcause`、`sepc`、`stval` 打印出来，再逐步判断条件。最终我采用了基于 `g_ufree_page` 的区间判定：当缺页地址落在 `naive_malloc` 下一页（尚未映射）的区间时，认为这是越界访问并直接报错退出；其余情况下再按栈扩展缺页的逻辑进行补页，保证递归过程中用户栈增长可以被正确修复。

这个实验里最容易犯的是“边界”的错误：`g_ufree_page` 在分配后会自增，因此越界访问触发的 `stval` 往往正好等于新的 `g_ufree_page`，如果判断条件写成 `stval < g_ufree_page` 之类会拦不住越界；反过来如果把判定范围写得过宽，又会误杀本该允许的栈增长缺页。我的做法是用两组输入验证：一组选择刚好不越界的 `n`，让程序能正确跑完并确认发生的是栈缺页且被修复；另一组选择越界的 `n`，确认输出固定错误并退出，且不会误触发栈补页分支。

收获上，我对“同样是 `page fault`，但语义完全不同”的理解更具体了：缺页异常只是硬件报告当前映射不满足访问，而该不该补页取决于内核对地址空间布局与内存策略的定义。建议方面，如果后续实验继续扩展，我认为不应只用 `g_ufree_page` 做粗粒度判断，而是引入更明确的 `VMA/region` 记录（每次分配登记允许访问区间），既能支持多次 `malloc`，也能避免把某些合法区域误判为非法。

实验三 进程等待和数据段复制(挑战实验3)

3.1 实验目的

lab3_challenge1 的实验目的，是通过完善 PKE 内核与系统调用机制，为用户态提供符合语义的 `wait(pid)`：父进程可在 `pid=-1` 时等待任意子进程退出、在 `pid>0` 时等待指定子进程退出，并在参数非法或目标并非其子进程时返回-1，从而同时实现“子进程退出后由父进程回收僵尸进程剩余资源”的资源回收机制，以及“父进程等待子进程结束后再继续执行”的进程同步能力；并配合 `fork` 的数据段复制/隔离，保证 `fork` 后父子进程数据段相互独立，使子进程对全局变量的写入不会影响父进程。

3.2 实验内容

3.2.1 进程等待

实验需要补全 `wait()` 的系统调用整体流程，并在 `syscall.c` 中实现对应的系统调用处理函数：`wait()` 接收参数 `pid`，用户态的 `wait()` 会通过系统调用入口进入内核并将处理交由 `sys_user_wait()` 统一完成，其具体逻辑由该函数负责实现：

```
// kernel entry point of wait. added @lab3_challenge1
ssize_t sys_user_wait(int pid) { return do_wait(pid); }
```

之所以交付给函数 `do_wait()`，是因为 `sys_user_wait()` 无法访问进程池数组。函数 `do_wait()` 位于 `process.c`，它可以访问进程池。当 `pid` 为-1 时父进程等待任意一个子进程退出并返回该子进程的 `pid`，当 `pid` 大于 0 时父进程仅等待进程号为 `pid` 的子进程退出并在其退出后返回该子进程的 `pid`，而当 `pid` 非法，或 `pid` 大于 0 但 `pid` 对应的进程并不是当前进程的子进程时应返回-1：

```
int do_wait(int pid) {
    int has_child = 0;
    if (pid == -1) {
        int idx = 0;
        while (idx < NPROC) {
            process *cp = &procs[idx];
            if (cp->parent == current) {
                has_child = 1;
                if (cp->status == ZOMBIE) {
                    cp->status = FREE;
                }
            }
            idx++;
        }
    }
    return pid;
}
```

```

        return idx;
    }
}
idx++;
}
if (!has_child) return -1;
insert_to_blocked_queue(current);
schedule();
return -2;
}
if (pid >= 0 && pid < NPROC) {
    process *cp = &procs[pid];
    if (cp->parent != current) return -1;
    if (cp->status == ZOMBIE) {
        cp->status = FREE;
        return pid;
    }
    insert_to_blocked_queue(current);
    schedule();
    return -2;
}
return -1;
}

```

BLOCKED 是内核专门用于表示“进程正在等待某事件完成”的状态，在这里用于实现类似 wait 的语义：当父进程需要等待某个子进程结束时，内核把父进程当前等待的目标进程号记录下来（例如存入 a0 寄存器或进程控制块中的对应字段），并将该寄存器/字段标记为有效的等待信息，然后把父进程状态切换为 BLOCKED，主动让出处理器并进入调度流程；之后父进程不再参与就绪队列的轮转，直到其等待的子进程退出时，父进程才会从 BLOCKED 转回 READY，并在后续调度中被重新运行。对于 pid = -1 的情况，还需要在阻塞之前先判断父进程是否确实存在子进程，否则不应无条件进入等待。紧接着的关键点在于“退出通知”机制：子进程在执行 exit 系统调用时，内核需要定位其父进程并检查父进程是否正因等待该子进程（或等待任意子进程）而处于 BLOCKED，一旦满足条件就将父进程从阻塞队列/等待状态中移出、更新其状态为 READY，并把它重新加入就绪队列，从而实现子进程退出时对父进程的唤醒与继续执行。

```

// implement the SYS_user_exit syscall
ssize_t sys_user_exit(uint64 code) {
    sprint("User exit with code:%d.\n", code);
    // reclaim the current process, and reschedule. added @lab3_1
    free_process( current );
    remove_block_and_insert(current);
}

```

```

    schedule();
    return 0;
}

```

上面出现的 `insert_to_blocked_queue()` 和 `remove_block_and_insert()` 这两个函数用来实现将进程移入等待队列或将进程移出等待队列并加入就绪队列。判断条件是根据 `process->parent` 与队列中的进程进行对比，这里不在赘述。

3.2.2 数据段复制

lab3 将进程地址空间简化为栈段、`trapframe` 段、系统段、堆段、代码段和数据段，在 `fork` 时不同段采取不同复制策略：`trapframe` 段和栈段在初始化时通常仅占用一个页面大小，因此 `fork` 过程中只需为子进程分配对应的物理页并将父进程相关物理页内容直接拷贝过去即可，从而完成 `trapframe` 与当前仍在使用的栈页面的复制；代码段由于不可写，不存在写时修改的问题，所以无需为子进程创建副本，只要把父进程的代码段物理页映射到子进程的虚拟地址空间实现共享即可；而数据段是可写的，`fork` 后父子进程必须互不影响，因此不能像代码段那样仅做 `mapping`，而需要在 `do_fork()` 中为子进程的数据段逐页申请新的物理内存并复制父进程对应页面内容，再建立子进程虚拟页到新物理页的映射，确保父子进程的数据段彼此独立。`do_fork()` 函数补充内容具体如下：

```

case DATA_SEGMENT:
    uint64 start_va = parent->mapped_info[i].va;
    int pages = parent->mapped_info[i].npages;
    for (int k = 0; k < pages; k++) {
        uint64 va = start_va + (uint64)k * PGSIZE;
        uint64 old_pa = lookup_pa(parent->pagetable, va);
        char *pa_copy = alloc_page();
        memcpy(pa_copy, (void *)old_pa, PGSIZE);
        map_pages(child->pagetable,
                  va,
                  PGSIZE,
                  (uint64)pa_copy,
                  prot_to_type(PROT_READ | PROT_WRITE, 1));
    }
    // after mapping, register the vm region (do not delete codes below!)
    child->mapped_info[child->total_mapped_region].va =
        parent->mapped_info[i].va;
    child->mapped_info[child->total_mapped_region].npages =
        parent->mapped_info[i].npages;
    child->mapped_info[child->total_mapped_region].seg_type =
        DATA_SEGMENT;
    child->total_mapped_region++;
    break;

```

3.2.3 实验结果

```
will fork a child from parent 1.
in alloc_proc. user frame 0x0000000087fa1000, user stack 0x000000007ffff000, user kstack 0x0000000087fa0000
do_fork map code segment at pa:0000000087fb2000 of parent to child at va:000000000010000.
going to insert process 2 to ready queue.
going to schedule process 2 to run.
Grandchild process end, flag = 2.
User exit with code:0.
going to insert process 1 to ready queue.
going to schedule process 1 to run.
Child process end, flag = 1.
User exit with code:0.
going to insert process 0 to ready queue.
going to schedule process 0 to run.
Parent process end, flag = 0.
User exit with code:0.
no more ready processes, system shutdown now.
System is shutting down with exit code 0.
```

3.2.4 日志 (git log)

```
huster@huster-VMware-Virtual-Platform:~/桌面/OS/riscv-pke$ git log
commit 8e0236d312eb385a76695d277a38398afcd2bd42 (HEAD -> lab3_challenge1_wait)
Author: Nuyoahwjl <3508529368@qq.com>
Date: Mon Dec 15 17:30:56 2025 +0800

    my work on lab3_challenge1 is done.

commit 2d991d7080dcfbdae29960dedbc335ed11b7a567
Merge: 18ef8d7 7f1f6a1
Author: Nuyoahwjl <3508529368@qq.com>
Date: Thu Dec 11 21:26:21 2025 +0800

    continue to work on lab3_challenge1
```

3.3 实验调试及心得

本实验的调试重点一半在“等待语义是否符合直觉”，另一半在“fork 后父子内存是否真正隔离”。在 wait 的实现中，我重点检查了三个状态转换：父进程在没有已退出子进程时必须进入 BLOCKED 并让出 CPU；子进程 exit 时需要能定位到父进程并把其从阻塞队列转回 READY；父进程被重新调度运行后应返回正确的子进程 pid（或在无子进程/参数非法时返回 -1）。为避免一次调试改动引入更多不确定性，我先只验证 do_wait 对 ZOMBIE 的扫描与 parent 关系判断正确，再验证 insert_to_blocked_queue/schedule 的阻塞行为，最后再把 exit 的唤醒逻辑串起来整体跑通。

调试 wait 时遇到的典型问题是：由于 schedule 之后控制流会回到父进程继续执行，若不重新检查一次子进程状态，父进程可能拿到一个中间态返回值而不是最终 pid。我的处理方式是在用户态 wait 包装里对被唤醒后的中间返回值做一次

重试（直到返回 ≥ 0 或 -1 ），从而保持对外语义一致。。

数据段复制部分，我主要用父子分别写同一全局变量再打印来验证隔离性：如果只是共享映射，父进程会观测到子进程的写入；如果逐页复制正确，则父子输出应彼此独立。实现上对 `DATA_SEGMENT` 采用逐页 `alloc_page + memcpy + map_pages` 的方式，保证子进程拿到的是新的物理页，并维持读写权限正确。

建议上，我认为实验可以提供一可视化调试输出（例如在 `schedule` 时打印 `READY/BLOCKED/ZOMBIE` 的数量变化），会降低同学们在队列操作与状态切换上的 `debug` 的难度。

实验四 相对路径(挑战实验 4)

4.1 实验目的

lab4_challenge1 的实验目的，是在 PKE 的文件系统中引入“当前工作目录（cwd）”语义，使内核能够基于 cwd 正确解析用户传入的相对路径，从而保证用户程序在调用 open/opendir 等所有需要路径参数的接口时，即便传入相对路径也能得到与绝对路径一致的正确行为；同时在用户态补全 pwd（显示当前工作目录）与 cd（切换当前工作目录）等基本命令接口，以支撑后续围绕“工作目录显示/切换/相对路径访问”的测试场景。

4.2 实验内容

4.2.1 显示当前目录 pwd

函数 pwd 由用户程序调用，可以打印当前工作目录。在提供的代码中，这一函数已经被转换成了系统调用。系统调用将当前目录写入以虚拟地址 pathva 开始的内存区域。最终函数被交付给 S 态下的 sys_user_rcwd() 函数：

```
ssize_t sys_user_rcwd(char *pathva) {
    char path[30] = {0};
    char *cwd_name = current->pfiles->cwd->name;
    if (cwd_name[0] != '/') {
        path[0] = '/';
        strcpy(path + 1, cwd_name);
    }
    else strcpy(path, cwd_name);
    char *dst = (char *)user_va_to_pa(current->pagetable, (void *)pathva);
    strcpy(dst, path);
    return 0;
}
```

sys_user_rcwd 系统调用用于把当前进程的工作目录路径（cwd）拷贝到用户态提供的缓冲区 pathva 中：它先从 current->pfiles->cwd->name 取出当前目录名，接着做一次规范化处理——如果目录名不是以 '/' 开头，就在前面补一个 '/'，从而确保得到一个看起来像绝对路径的字符串并存入内核栈上的 path[30]；随后通过 user_va_to_pa(current->pagetable, pathva) 将用户虚拟地址转换为当前进程页表下对应的物理地址得到 dst，最后用 strcpy 把拼好的路径复制到用户缓冲区。

4.2.2 切换目录 cd

函数 `cd` 由用户程序调用，可以切换当前目录到另一个目录。在提供的代码中，这一函数已经被转换成了系统调用。系统调用被交付给下面的函数：

```
ssize_t sys_user_ccwd(char *pathva) {
    char *user_pa = (char *)user_va_to_pa(
        (pagetable_t)(current->pagetable), pathva );
    char path[30] = {0};
    struct dentry *cwd = current->pfiles->cwd;
    if (user_pa[0] == '.') {
        if (user_pa[1] == '/') {
            strcpy(path, user_pa + 2);
            cwd = hash_get_dentry(cwd, path);
        } else if (user_pa[1] == '.') {
            cwd = cwd->parent;
            if (user_pa[2] == '/') {
                strcpy(path, user_pa + 3);
                cwd = hash_get_dentry(cwd, path);
            }
        }
    } else {
        const char *start = (user_pa[0] == '/') ? user_pa + 1 : user_pa;
        strcpy(path, start);
        cwd = hash_get_dentry(cwd, path);
    }
    current->pfiles->cwd = cwd;
    return 0;
}
```

`sys_user_ccwd` 系统调用用于根据用户传入的路径字符串切换当前进程的工作目录（`cwd`）：它先把用户虚拟地址 `pathva` 转成内核可直接访问的地址，然后以当前 `cwd` 为起点解析路径——若路径以 `./` 开头，就将后续子路径复制到 `path` 并在当前目录下查找目标目录；若以 `../` 开头，则先把 `cwd` 移到父目录，并在存在 `../xxx` 的情况下继续解析 `xxx`；否则（绝对路径或普通相对路径），会去掉可能存在的前导 `/`，把剩余部分作为路径在当前 `cwd` 下查找对应目录项；最终把解析得到的 `cwd` 写回 `current->pfiles->cwd` 完成工作目录更新，并返回 `0` 表示成功。

4.2.3 修改 `sys_user_open`

除了 `pwd` 和 `cd`，`open` 函数现在也能使用相对路径了。因此，我们必须对 `open` 函数的底层函数调用进行修改，从而支持相对路径文件的打开。

```
ssize_t sys_user_open(char *pathva, int flags) {
    char *user_pa = (char *)user_va_to_pa(
```

```

    (pagetable_t)(current->pagetable), pathva );
char path[30] = {0};
char *cwd_name = current->pfiles->cwd->name;
if (user_pa[0] == '.') {
    if (user_pa[1] == '/') {
        path[0] = '/';
        strcpy(path + 1, cwd_name);
        strcat(path, user_pa + 1);
    } else if (user_pa[1] == '.') {
        path[0] = '/';
        strcpy(path + 1, current->pfiles->cwd->parent->name);
        if (user_pa[2] == '/') {
            strcat(path, user_pa + 3);
        }
    } else strcpy(path, user_pa);
} else strcpy(path, user_pa);
return do_open(path, flags);
}

```

底层函数调用落实到 `sys_user_open()` 函数。由于相对路径到绝对路径的转换已经实现了，直接将其加入 `sys_user_open()` 函数即可。最后将拼好的 `path` 传给底层 `do_open(path, flags)` 执行真正的打开操作并返回其结果。

4.2.4 实验结果

```

===== Test 1: change current directory =====
cwd:/
change current directory to ./RAMDISK0
cwd:/RAMDISK0

===== Test 2: write/read file by relative path =====
write: ./ramfile
file descriptor fd: 0
write content:
hello world
read: ./ramfile
read content:
hello world

===== Test 3: Go to parent directory =====
cwd:/RAMDISK0
change current directory to ..
cwd:/
read: ./hostfile.txt
file descriptor fd: 0
read content:
This is an apple.
Apples are good for our health.

All tests passed!

```

4.2.5 日志 (git log)

```
huster@huster-VMware-Virtual-Platform:~/桌面/OS/riscv-pke$ git log
commit 5ad2dd5d7397cc725cdd272b8829cfc5922386f1 (HEAD -> lab4_challenge1_relativepath)
Author: Nuyoahwjl <3508529368@qq.com>
Date:   Mon Dec 15 19:12:00 2025 +0800

    my work on lab4_challenge1 is done.

commit 925b4917382ab2327cbe12b5e99de101fd7c3dc0
Merge: b047d4e 499b62a
Author: Nuyoahwjl <3508529368@qq.com>
Date:   Mon Dec 15 17:32:25 2025 +0800

    continue to work on lab4_challenge1
```

4.3 实验调试及心得

相对路径实验的难点不在字符串拼接，而在语义一致性：`pwd` 要输出规范化路径，`cd` 要正确维护 `cwd` 并支持 `./` 与 `..` 的解析，`open` 等依赖路径的系统调用也必须以同样的解析规则得到同一个目标目录项，否则会出现 `pwd` 显示已切换，但 `open` 仍按旧 `cwd` 解析的错误。我的调试策略是先把 `rcwd/ccwd` 单独跑通，确保 `current->pfiles->cwd` 的更新是正确的，再改 `sys_user_open`，让文件打开复用相同的 `cwd` 语义。

调试中我重点关注两个边界条件：其一是根目录与父目录处理，`cd ..` 在根目录时是否会把 `cwd` 变成空指针或非法目录项；其二是路径缓冲区长度与拼接顺序，特别是 `sys_user_open` 中把 `cwd_name` 与用户相对路径拼成绝对路径时，是否会漏掉分隔符或多出重复的 `'/'`。在 `sys_user_open` 的实现里，我按路径是否以 `'/'` 开头分别处理 `./` 与 `..`，并在需要时显式补上前导 `'/'` 与 `cwd_name`，再把后续子路径拼接进去，最后交给 `do_open` 执行真正的打开逻辑。

收获上，我更直观地理解了 `cwd` 本质是进程文件上下文的一部分，只有把它作为内核状态维护并被所有路径相关系统调用一致使用，用户态的 `shell` 命令（`cd/pwd`）才有意义。建议方面，如果课程允许扩展，我认为可以进一步加入更通用的路径归一化（例如处理多级 `../`、重复斜杠等）并配套更严格的边界检查（路径长度、根目录 `parent` 保护）。